We promise that this tutorial is not a permanent document. We will continuously revise and optimize the tutorial based on feedback from everyone on the forum and actual development experience

Part 1: Customized Linux system using Petalinux

Special reminder, the environment used in this tutorial is built according to the tutorial "Xilinx Development Environment Installation Tutorial". Please follow this tutorial to build the Development Environment first. (Try to upgrade the VM to the latest version, Ubuntu16 is compatible with vivado2017 version, Ubuntu18 is compatible with vivado2020, please try to use Ubuntu18.) This tutorial uses Ubuntu18.

This tutorial refers to the Xilinx official tutorial " ug1144-petalinux-tools-reference-guide ", using Petalinux to build a basic Linux system.

1.

Petalinux build project

1.

Create a working directory under the user directory. "~" represents the user directory. For example, when our username is "alinx", "~" is equivalent to the path "/home/alinx". The peta_prj path is used to store all petalinux projects, the linuxPsBase path is the path where the petalinux project used in this chapter is stored, and the hardware path is used to store the hardware description files needed for the petalinux project used in this chapter.

```
ıkdir -p ~/peta_prj/linuxPsBase/hardware
```

2.

Copy the file with the extension xsa exported from the vivado project (which is the hardware description file mentioned in step 1) to the hardware folder just created (the vivado project is in the folder corresponding to the chapter of the basic tutorial source code path).

Note: The xsa file exported from the vivado project in the corresponding chapter path of this tutorial source code package is used here. You can also use the ps_hello project in the "Vitis Application Tutorial" source code package.

3.

Enter the path where the petalinux project is stored (in step 1).

```
d ~/peta_prj/linuxPsBase/
```

4.

Set petalinux environment variables, run the following command (environment variables are only valid in the currently open end point, and the commands related to petalinux should be entered in the current end point. If the current end point is closed, you need to run this command again) .

```
ource /opt/pkg/petalinux/settings.sh
```

5.

Use the command "petalinux-create" to create a petalinux project. The "-t" parameter refers to the type that needs to be created. Here we are creating the project, so "project" follows it. "-n" refers to the project name as "petalinux", and the "--template" parameter refers to using the "zynqMP" template to create the project.

petalinux-create -t project -n petalinux --template zynqMP 6.

Use the following command to enter the petalinux project directory.

```
d ~/peta_prj/linuxPsBase/petalinux
```

7.

Use the "petalinux-config" command with the "--get-hw-description" parameter to configure the hardware information of the petalinux project. The path where our hardware description file (xsa file) is located after the "--get-hw-description" parameter is the path created in step 1. The two points in "../hardware/" refer to the parent path of the current working path, which is equivalent to "~/peta_prj/linuxPsBase/hardware" in the current step. Note: There must be only one xsa file in the hardware information directory (currently "../hardware/").

```
etalinux-config --get-hw-description ../hardware/
```

If this error occurs, before executing this command, make the Ubuntu interface full screen and make the end point command box full screen.

Note: This tutorial follows the usage habits of the old version of petalinux and configures the "kernel autoconfig" and "u-boot autoconfig" options, but these two options can also be left unselected. If these two options are configured in the project and compile errors occur, you can also try to remove them and compile again.

9.

If "Save" was not selected in the previous step, when exiting the configuration interface, the interface of whether to save the configuration will pop up. Simply select < Yes >.

1.

Copy the effect package to the virtual machine and unzip it. Here I unzip it to the/home/Downloads/petalinux_offline_pkg path, as shown below:

2.

Open the end point in the petalinux project and enter the command (if there is an error, try to make Ubuntu and the command box full screen)

```
etalinux-config
```

Then the following configuration interface will pop up:

Note that if compile reports an error prompt "Network access disabled through BB_NO_NETWORK" later, uncheck "Enable BB NO NETWOTRK" and try compiling again.

5.

Set the path of the "sstate_aarch64" package and select the "Local sstate feeds settings --- >" option

Press Enter directly in the following interface.

Note: The path in the picture is not fully displayed . After setting, press Enter to select the < ok > option.

The content in the path is as follows:

6.

To set the path of the "downloads" package, first select the < Exit > option to return to the interface shown in the figure below, and then select the "Add pre-mirror url --- >" option

The content in the path is as follows:

7.

Select the < Save > option, and then select the < Exit > option to exit the configuration interface.

3.

Set local kernel source code and u-boot source code

If there is a need to modify the kernel source code and u-boot source code, we can set to use the local kernel source code and u-boot source code.

The effect package we provide also includes the kernel source code and u-boot source code of xilinx, which can be used directly. The path is as follows:

2.

Open end point in petalinux project and enter command

```
etalinux-config
```

Then the following configuration interface will pop up:

3.

Set the path of the kernel source code, select the "Linux Components Selection --- >" option,

Then select the < Save > option and select the < Exit > option to exit the configuration interface.

4.

Modify device tree, compile petalinux project

1.

In the path of the folder in the above figure, "Home/peta_prj_linuxPsBase" is the path we created using the mkdir command. The path starting from "petalinux" and the files in the path are generated by the petalinux-create command. We enter the path of/home/alinx/peta_prj/linuxPsBase/petalinux/project-spec/meta-user/recipes-bsp/device-tree/files. The two files with the dtsi suffix here are custom device tree files generated by petalinux for us. We can edit these two device trees according to our own needs. We can edit either one or both of them. In this tutorial, we choose to edit the system-user.dtsi file and double-click to open it and modify it to the following content:

```
nclude/ "system-conf.dtsi"

{









 SD */

sdhci1 {

isable-wp;

o-1-8-v;
```

```
 USB */

dwc3_0 {

tatus = "okay";

r_mode = "host";
```

- petalinux-config -c u-boot
- petalinux-config -c kernel
- petalinux-config -c rootfs

5. Compile petalinux project:

-

petalinux-build

6.

Manually modify the petalinux configuration (these configuration files can be directly copied to other projects for use):

- Various manually configurable files are saved in the "< plnx-project-root >/project-spec/meta-user" directory
- The file "< plnx-project-root >/project-spec/meta-user/recipes-bsp/u-boot/files/platform-top.h" is used to modify the u-boot configuration
- The "system-user.dtsi" and "pl-custom.dtsi" files in the path "< plnx-project-root >/project-spec/meta-user/recipes-bsp/device-tree/files/" are used to customize the device tree
- The configuration of the kernel is saved in the path < plnx-project-root >/project-spec/meta-user/recipes-kernel/linux/linux-xlnx/
-

< plnx-project-root >/project-spec/meta-user/project-spec/meta-user/recipes-apps/"The app is saved in the path

7.

Clean up petalinux:

-

If there are various problems with the petalinux compile, especially if we cannot compile normally after modifying some configurations or keep stuck, we can try using the command " petalinux-build -x mrproper -f " to clean up the previous compile results.

8.

Creating and using BSP packages

By using the bsp package of petalinux, we can easily and quickly provide our verified petalinux project to others for use, or use the verified petalinux project of others. Only two commands are needed to create and use the bsp package.

1.

Create bsp package

Use the following command to create the bsp package of the petalinux project into the path of the currently open end point:

```
etalinux-package --bsp -p ./petalinux/ --output chapter1.bsp
```

- The path after the -p parameter is the first-level path of the petalinux project
- The output parameter is followed by the name of the bsp package

Note: If the project that created the BSP package can be successfully compiled, then the project created using the BSP package can also be directly compiled.

Part 2: Add startup script and user files

Using petalinux to create Linux, the default root file system is ramdisk type, this type of file system is read-only type, any modification to the file system will be lost after the system restarts. This chapter describes how to add user applications and data files to this file system and how to automatically start user applications at startup.

1.

Preparation work

Using the petalinux project created in Chapter 1, open an end point in the project path and set up environment variables.

2.

Self-starting script

1.

Enter the following command to create an application named "autostart" using the "install" template.

```
etalinux-create -t apps --template install --name autostart --enable
```

4.

Return to the previous directory and edit the file autostart.bb as follows:

This file is the autostart recipe.

UMMARY = "Simple autostart application"

ECTION = "PETALINUX/apps"

ICENSE = "MIT"

IC_FILES_CHKSUM =

ile://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

RC_URI = "file://autostart \

```
= "${WORKDIR}"


nherit update-rc.d


NITSCRIPT_NAME = "autostart"

NITSCRIPT_PARAMS = "start 99 5 ."


o_install() {

nstall -d ${D}${sysconfdir}/init.d
```

```
ıstall -m 0755 ${S}/autostart ${D}${sysconfdir}/init.d/autostart

:DEPENDS_${PN}_append += "bash"
```

The main function is to copy autostart to the root file system init.d directory, and then add 0755 permissions. RDEPENDS_ ${PN} _append += "bash" means this depends on bash.

7.

Eject the SD card, connect the serial port, and use this SD card to start the development board.

After the system start-up, it first prints "Hello ALINX World" and then outputs the content printed in the script we added to the SD card. We can modify the script added to the SD card and add custom content, which will not be restored by the system restart.

Part 3: Program hello world

1.

Program code

1.

Create a directory "app_helloworld" in the path "/home/alinx/peta_prj/linuxPsBase" to store the application in this chapter. As shown below:

If using Vitis' arm compiler, use the following command to set environment variables.

```
ource /tools/Xilinx/Vitis/2020.1/settings64.sh
```

Note: Environment variables are only valid at the current end point, and subsequent commands should be completed at the current end point.

2.

Use the following command to cross-compile the program and generate an executable program on the board, still named "helloworld".

```
arch64-linux-gnu-gcc -o helloworld main.c
```

3.

Use the linux system start-up development board customized by petalinux in Chapter 1, and copy the executable program helloworld to the SD card together.

4.

Connect the serial port, set the card to start from the SD card, and then power it on.

5.

Enter your account and password to access the Linux system.

6.

Use the following command to search for the path where the "helloworld" program is located.

```
nd / -name helloworld
```

```
rite(lGpioFd, "1", 2);
```

lse

```
rite(lGpioFd, "0", 2);
```

```
lose(lGpioFd);
```

```
nt main()

ed_ctl_init();

rintf("================start led ctrl==============\n");

while(1)

ed_ctl_on(true);

rintf("on\r\n");

sleep(500000);
```

```
ed_ctl_on(false);

rintf("off\r\n");

sleep(500000);

eturn 0;
```

The main function of the "led_ctl_init () " function in the code is to export the GPIO pin and initialize it as an output pin. The "led_ctl_on () " function controls the high and low levels of the pins to achieve the on and off of the LED light.

3.

Set environment variables with the following command and compile the code:

```
ource /tools/Xilinx/Vivado/2020.1/settings64.sh & aarch64-linux-gnu-gcc -o gpio_test gpio_test.c
```

7.

Check Petalinux Package Groups → packagegroup-petalinux-v4lutils to configure the v4l tool, which is mainly a Linux video configuration tool that may be used in video development.

8.

Check PetalLinux Package Groups → packagegroup-petalinux-x11 Configure x11 Library

9.

Check Filesystem Packages → libs → libmali-xlnx to configure GPU library

10.

Click the hammer icon in the lower left corner to compile the program.

11.

In the build - qt_test - ZYNQ MP - Debug directory, you can see that a qt_test file is generated to run on the board.

2.

Go to the "Image Packaging Configuration --- > Root filesystem type" option and select EXT (SD/eMMC/QSPI/SATA/USB).

3.

Select the "Device node of SD device" option to set the root file system path. This is crucial. If you fill it incorrectly, you cannot start the Linux. Mmcblk represents the SD card and emmc. There is a default value of "mmcblk0p2", which represents the second partition of the first mmc device. If you use the vivado project named "ps_hello" according to the tutorial , then our SD card is the second mmc device. Fill in /dev/mmcblk1p2 here.

4.

(Note that in the chapter "QT and OPENCV cross-compile Development Environment", in order to build the SDK, this petalinux project is set to allow online compilation. If necessary, remember to restore the setting to offline compilation.) Then save the settings and exit the configuration interface, compile the petalinux system, and package BOOT.bin.

2.

Creating an SD card file system

Creating an SD card file system will result in the loss of content on the SD card. Please make a backup first.

SD card modification partition

1.

Insert the SD card of the development board into the card reader, and then insert it into the USB port of the computer. If the card reader is not connected to the virtual machine Linux, refer to the method in the figure below.

2.

In the search path of Ubuntu, enter disk and the Disks icon will appear.

3.

Click on the Disks icon with the mouse, and the "Disks" dialog box will appear. The SD card here is divided into two partitions, one named FAT and the other named EXT. However, regardless of the original SD card partition, it needs to be re-partitioned here.

4.

Select the Delete Partition icon below each partition to delete all partitions.

5.

Click the Add Partition icon to add the first partition. For this experiment, fill in 5000MB in FAT format for

6.

Create a second partition to store the root file system in EXT4 format and named EXT.

7.

After creating the partition, reinsert the SD card.

8.

The system will automatically mount the partition and pop up a window

Make an SD card to start Linux

1.

Enter the following decompression command to extract the root file system to the SD card EXT4 partition. Please note the format of the decompression command, which must be consistent with the tutorial. "/media/alinx/EXT" is the path of the SD card EXT partition in the current virtual machine environment. If you do not follow the tutorial completely, there may be differences. Please modify it according to your actual

situation. The petalinux project path "< petalinux path >" requires you to complete all the paths in front of your images by yourself.

```
udo tar -zxvf <petalinux path>/images/linux/rootfs.tar.gz -C /media/alinx/EXT
```

2.

Type "sudo sync" in the command window to synchronize cached data. Synchronization may take a long time.

3.

Copy BOOT.bin, boot.scr, and image.ub to the FAT partition of the sd.

4.

Set the development board to start in SD mode and start the development board. At this point, the system will be restarted without losing applications and data.

Part 9: Vitis accelerates basic platform creation

Note: This tutorial requires proficiency in using vivado and petalinux. It is also recommended to learn OpenCL and HLS before trying Vitis acceleration.

This chapter refers to the official xilinx routine:

https://github.com/Xilinx/Vitis-Tutorials/blob/2020.1/Vitis_Platform_Creation/Introduction/02-Edge-AI-ZCU104/step1.md#start-of-content.

This chapter mainly introduces how to customize the software and hardware platform using PL-side FPGA acceleration, which requires special processing in the Vivado project to be used for acceleration. Acceleration requires knowledge of OpenCL and HLS.

1.

Vivado hardware platform

We modify the vivado project named "ps_hello" (using the "ALINX_ZYNQ_MPSoC Development Platform Vitis Application Tutorial" tutorial corresponding to the source code package or the source code package of this tutorial).

1.

Open the vivado project

2.

Expand all paths in the Source window and double-click the design_1_i: design_1 (design_1.bd) tab to open the design interface

3.

Double-click "zynq_ultra_ps_e_0" in the "Diagram" interface to open the interface on the PS side

4.

Select the "PS-PL Configuration" tag, change the value after the "General > Interrupt > PL to PS > IRQ0 [0-7]" tag to 1, and then click OK.

5.

Delete the connection in the following picture, select the connection, and then press the "Del" key on the keyboard

6.

Add a clock IP

Then double-click on this ip to open the ip configuration interface. Configure according to the figure below, and then click the ok button. 7.

Add three "Processor System Reset" ips

8.

Connect as shown in the figure (Note: the connection here must be exactly the same as in the figure, and the three identical IP and clock IP pins should be connected)

9.

Click the "Window" button on the menu bar and select the "Platform Interfaces" option

Then the window in the upper left corner will change to the state shown in the figure below, click on the blue text inside Afterwards, this window will become the state shown in the figure below Scroll down to find "clk_wiz_0", then double-click "clk_out1", "clk_out2", "clk_out3" to enable these clocks, and the front icon will change from gray to blue Then click "clk_out2" and set "clk_out2" as the default clock in the "Source File Properties" window below, as shown below

Modify the IDs of the three clocks according to the following figure, "clk_out2" id is set to 0 , "clk_out1" id is set to 1, and "clk_out3" id is set to 2. Note that after changing the ID numbers, be sure to press Enter to confirm.

10.

Add an interrupt controller IP

Then click "Run Connection Automation" to automatically rout ePop up this interface and click ok directly. Then click the button to organize the layout, and it will become the state as shown in the figure below. Expand the "interrupt" pin of the end point controller IP Connect the unfolded "irq" according to the picture, and then organize the layout 11.

Enter this command in the Tcl window below

```
et_property PFM.IRQ {intr {id 0 range 32}} [get_bd_cells /axi_intc_0]
```

12.

Then go back to the "Platform" window and enable the port as shown in the figure

13.

Modify "memport" of "S_AXI_HPC0_FPD" and "S_AXI_HPC1_FPD" ports to "S_AXI_HP"

14.

Enable M01_AXI in ps8_0_axi_periph (AXI Interconnect2.1) to M08_AXI and press Crtl + S to save your changes

15.

Click "Tools" in the menu bar and select the first option "Validate Design".

Waiting for completion The interface in the following picture pops up, with a warning to ignore it, click OK. 16.

In the Source window, right-click design_1_i: design_1 (design_1.bd) and select the Create HDL Wrapper option

Pop up the following interface and click OK directly.

Then the following pop-up window will appear. There is also this warning, don't worry about it, just click OK. 17.

Click "Generate Block Design" in the left menu bar.

Then a pop-up window as shown in the figure below will pop up. Select "Global" and click "Generate". After completing the pop-up window, simply click OK without worrying about the warning. 18.

Click "File > Export > Exoprt Hardware" in the menu bar to export the hardware description file

Pop up the following interface and click "Next" directly. The next interface is modified according to the figure , click "Next" Click finish 2.

Petalinux custom system

1.

Create a new path "/home/alinx/peta_prj/linux4acc" and create a petalinux project in this path.

```
etalinux-create -t project -n petalinux --template zynqMP
```

2.

Create a new "hardware" folder in the path "/home/alinx/peta_prj/linux4acc" and copy the ".xsa" hardware description file created in section "9.1".

3.

Go back to the end point window, cd into the petalinux project path, and set the hardware description file.

After the configuration interface pops up, set it to boot from the SD card, as shown below.

If necessary, set up offline compile, refer to Chapter 1, the operation will not be repeated here. Then save the configuration and exit the configuration interface.

4.

Go to the "project-spec/meta-user/conf" path of the petalinux project and open the "user-rootfsconfig" file.

Modify to the following content.

Note: Mention Each package in individual line

These packages will get added into rootfs menu entry

:ONFIG_gpio-demo

:ONFIG_peekpoke

Packages for base XRT support:

:ONFIG_packagegroup-petalinux-xrt

:ONFIG_xrt-dev

Packages for easy system management

:ONFIG_dnf

:ONFIG_e2fsprogs-resize2fs

:ONFIG_parted

Packages for Vitis-AI dependencies support:

:ONFIG_packagegroup-petalinux-vitisai

Packages for natively building Vitis AI applications on target board:

:ONFIG_packagegroup-petalinux-self-hosted

:ONFIG_cmake

:ONFIG_packagegroup-petalinux-vitisai-dev

CONFIG_xrt-dev

:ONFIG_opencl-clhpp-dev

:ONFIG_opencl-headers-dev

:ONFIG_packagegroup-petalinux-opencv

:ONFIG_packagegroup-petalinux-opencv-dev

Packages for running Vitis-AI demo applications with GUI#

:ONFIG_mesa-megadriver

:ONFIG_packagegroup-petalinux-x11

:ONFIG_packagegroup-petalinux-v4lutils

```
:ONFIG_packagegroup-petalinux-matchbox
```

5.

Enter the following command in the console to configure the root file system.

```
etalinux-config -c rootfs
```

Select all options in "user packages --- >".

6.

The configuration in "Image Features --- >" is shown in the following figure

1.

If there is a compile error later , select the two options in the apps, then save and exit,

and then compile. (This step can be unchecked first)

2.

Save and exit to the Console.

9.

Enter the following command in the Console to configure the kernel. It may take some time for the configuration interface to pop up.

```
etalinux-config -c kernel
```

Remove all the options in "CPU Power Management --- >" as shown below.

Then save and exit the configuration interface.

10.

Open the path "project-spec/meta-user/recipes-bsp/device-tree/files" in the petalinux project, modify the "system-user.dtsi" file inside, modify the device tree, and the content is as follows

nclude/ "system-conf.dtsi"

{

hosen {

ootargs = "earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/mmcblk1p2 rw

ootwait cma=512M";

amba {

yxclmm_drm {

ompatible = "xlnx,zocl";

```
tatus = "okay";

ıterrupt-parent = <&axi_intc_0>;

ıterrupts = <0 4>, <1 4>, <2 4>, <3 4>,

4 4>, <5 4>, <6 4>, <7 4>,

8 4>, <9 4>, <10 4>, <11 4>,

12 4>, <13 4>, <14 4>, <15 4>,

16 4>, <17 4>, <18 4>, <19 4>,

20 4>, <21 4>, <22 4>, <23 4>,

24 4>, <25 4>, <26 4>, <27 4>,

28 4>, <29 4>, <30 4>, <31 4>;
```

```
:axi_intc_0 {

lnx,kind-of-intr = <0x0>;

lnx,num-intr-inputs = <0x20>;

ıterrupt-parent = <&gic>;

ıterrupts = <0 89 4>;

:sdhci1 {

o-1-8-v;
```

```
isable-wp;
```

11.

Use the following command to compile the petalinux project

```
etalinux-build
```

12.

to build the SDK, pay attention to the need to allow petalinux projects to compile online , and the method of building the SDK is the same as that in the chapter "QT and OPENCV cross-compile Development Environment". You can also use the one we have compiled. In the corresponding chapter directory of the corresponding source code package in this chapter, note that the SDK is not the same as that in the section "QT and OPENCV cross-compile Development Environment".

petalinux-build --sdk 13.

Create a new pfm folder in the directory of the petalinux project, and install the SDK we just built into this folder.

Open end point in the "images/linux" path of the petalinux project and enter the following command to install the sdk.

```
sdk.sh -d ../../pfm/
```

14.

Create two new folders "image" and "root" in the pfm folder, as shown below.

Copy the four .elf files from the "images/linux" path of the petalinux project to the root folder here.

Rename "zynqmp_fsbl" to "fsbl.elf".

Then create a new "linux.bif" file in the r

he root folder.

Open this file and enter the following content.

```
 linux */

ıe_ROM_image:



sbl_config] a53_x64

ɔootloader] <fsbl.elf>

ɔmufw_image] <pmufw.elf>

destination_device=pl] <bitstream>
```

```
destination_cpu=a53-0, exception_level=el-3, trustzone] <bl31.elf>

destination_cpu=a53-0, exception_level=el-2] <u-boot.elf>
```

15.

Copy "boot.scr" and "system.dtb" from the "images/linux" path of the petalinux project to the image folder.

3.

Vitis creates acceleration platform

1.

Open an end point in the pfm directory and enter the following command to create a new vitis project.

```
ource /tools/Xilinx/Vitis/2020.1/settings64.sh && vitis -workspace wksp1
```

2.

After opening the vitis interface, select "Create Platform Project" on the welcome page.

3.

Vitis creates acceleration platform

1.

Open an end point in the pfm directory and enter the following command to create a new vitis project.

```
ource /tools/Xilinx/Vitis/2020.1/settings64.sh && vitis -workspace wksp1
```

2.

After opening the vitis interface, select "Create Platform Project" on the welcome page.

In the pop-up window, fill in "Project name" with "15eg_base", and then click next.

Select the first option on the next interface, and then click next.

Click the "Browse" button on the next interface to select the XSA file.

Select the "15eg_base" obtained from the vivado project in this chapter.

Then the interface will have more options. We change "Operating system" to "Linux" and click the Finish button.

3.

Afterwards, the vitis project will be opened. Click on the contents in the window in the order shown in the picture, and select "Bif file".

Select the newly created'linux.bif '.

In this window, select the path of "Boot Components Directory".

Just select the previously created root path.

Then select the image path.

4.

Click the "Little Hammer" icon in the upper left corner of Vitis to compile the project.

4.

Create an application testing acceleration platform

1.

After compiling successfully, select "File > New > Application Project..." in the menu bar to create an application testing acceleration platform.

Click next directly on the interface shown in the following picture.

2.

The next interface doesn't need to be modified, just go straight to next. The "15eg_base [custom]" here is the acceleration platform just compiled.

3.

Next interface Enter the project name "hello_world", then next.

4.

In the next interface, we need to select two paths, "Sysroot path" and "Kernel image", as shown in the following figure.

5.

Then we need to choose an engineering template, but the template we need is not available here. Click the "Vitis IDE Examples" button in the lower left corner of the interface, and then click download to download.

6.

After the download is completed, there will be more content as shown in the following figure. Click OK. If the download fails in the vitis software, you can also go to the link given next to it to download it manually, and then put it in the given path. We also put the downloaded official routine package in the corresponding chapter folder of the corresponding source code package of this chapter. If you want to download it by yourself, you must pay attention to the version 2020.1. If you are not familiar with the use of GitHub, it is recommended to use the package we provide directly.

7.

The content in the "Vitis IDE Libraries..." button in the lower left corner also needs to be downloaded.

8.

Then in the interface of selecting routines, there are also some more routines. We select "Hello World (HLS C/C++ kernel) " and then click "Finish".

9.

Select "Hardware" from the drop-down menu next to the hammer to compile the application. It should be noted that if the virtual machine memory is not enough, the compile will fail. In this tutorial, 16G of memory is allocated to the virtual machine to compile successfully.

If the following error occurs, you need to apply the patch of the vitis software (named "y2k22_patch-1.2.zip" in the corresponding source code package of this chapter), and the installation method refers to the "README" in the patch Compressed Packet.

Compile successful log as shown below.

10.

After compiling successfully, go to the path in the figure below and copy all the files inside to the FAT partition of SD.

11.

Extract "rootfs.tar.gz" from the "images/linux" path in the petalinux project to the EXT partition of the SD card according to the method in the "SD card root file system" chapter.

5.

Run program test acceleration

1.

Start the development board using the SD card made in the previous step. After system start-up, enter the following command to enter the first partition of the SD card.

```
d /media/sd-mmcblk1p1
```

2.

Before running the test program, set the environment variables with the following command.

```
xport XILINX_XRT=/usr
```

3.

Run the application using the following command.

```
```

```
hello_world vadd.xclbin
```

If "TEST PASSED" is printed on the serial port, it means that the acceleration platform has been successfully created.

Part 10: Starting Linux from QSPI Flash

1.

Create a new folder "linuxQflashStart" in the path "/home/alinx/peta_prj", open the end point in this folder, use the following command to create a petalinux project, and create

a hardware folder to store hardware description files (still using the vivado project named ps_hello).

```
etalinux-create -t project -n petalinux --template zynqMP
```

2.

Enter the petalinux folder in the end point, use the following command to configure the hardware description file of the petalinux project, open the configuration interface, do not configure anything directly exit.

```
```

```
etalinux-config --get-hw-description ../hardware/
```

3.

Use the petalinux-config command to configure the petalinux project. Select primary flash in the Subsystem AUTO Hardware Settings --- > Advanced bootable images storage Settings --- > boot image settings --- > image storage media interface.

4.

Select primary flash in Subsystem AUTO Hardware Settings --- > Advanced bootable images storage Settings --- > kernel image settings --- > image storage media option.

1.

Subsystem AUTO Hardware Settings --- > Flash Settings, follow the figure to modify the partition size of QSPI flash.

•

The boot partition stores .bit files describing FPGA, u-boot, fsbl, device tree.dtb and other files. The size of the boot partition needs to be larger than the sum of these file sizes and needs to be modified according to the actual situation.

-

Bootenv partition stores environment variables for u-boot, modify according to actual situation.

-

The kernel partition stores image.ub, and the partition should be larger than image.ub. It needs to be modified according to the actual situation.

-

The bootscr partition stores boot.scr, and the partition should be larger than boot.scr. It needs to be modified according to the actual situation.

-

In addition, it should be noted that the total size of all packaged files cannot exceed the size of Flash.

6.

Select INITRAMFS in Image Packaging Configuration --- > Root filesystem type to use a RAM type root file system, which makes it easy to package and burn to QSPI Flash.

In Firmware Version Configuration --- >, you can modify the Host name and other information. Here, you can modify it as needed. Keep it as it is in the tutorial.

Set offline compile as needed. The configuration of the "Yocto Settings" interface is as follows. After saving, exit the configuration interface of petalinux-config. 9.

Use the petalinux-config -c u-boot command to configure u-boot. After opening the configuration interface, enter the ARM architecture --- > Boot script offset interface and set the offset address of the bootscr partition.

Offset address = boot size + bootenv size + kernel size.

0x1F10000 = 0xA00000 + 0x10000 + 0x1500000。

Save and exit the configuration interface after setting is completed.

10.

Open project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr.bbappend under the petalinux project path. Modify the kernel offset address and kernel size to correspond to the settings in "Subsystem AUTO Hardware Settings --- > Flash Settings" of the petalinux-config command.

11.

After saving, use the petalinux-build command to compile the project.

12.

After compiling successfully, use the following command to synthesize BOOT.bin, and use the --kernel option to compile the kernel into BOOT.bin. Use the --add option to compile boot.scr into the kernel. Note that after boot.scr, you need to set the offset address through the --offset option, which corresponds to the setting in step 8. Because there is no PL side in the vivado project, there is no -fpga in the parameter.

```
etalinux-package --boot --fsbl ./images/linux/zynqmp_fsbl.elf --u-boot --kernel --add
images/linux/boot.scr --offset 0x01F10000 --force
```

13.

Refer to the method in the vitis tutorial to burn BOOT.bin to the development board.

14.

Set the DIP switch on the development board to QSPI startup before powering on the development board.

Part 11: NVMe SSD operation under Linux

Note: When using an SSD, do not touch the surface of the SSD to avoid electrostatic damage to the device, which may cause the SSD to malfunction.

1.

Petalinux configuration

1.1

Create a new petalinux project

Configure the petalinux project using the xsa file used in chapter 1 (the ps_hello project in the vitis tutorial

source package). Set to mount the root file system from the SD card (refer to "Chapter 8 SD Card Root File System" in this tutorial for specific methods).

Modify the device tree.

```
nclude/ "system-conf.dtsi"

{









 SD */

sdhci1 {

isable-wp;

o-1-8-v;
```

```
 USB */

dwc3_0 {

tatus = "okay";

r_mode = "host";
```

1.2

Configure Linux kernel

SSDs using the M.2 interface require the kernel to load PCIe and NVME drivers. PCIe drivers are loaded by default and NVME drivers also need to be configured. Enter the following command in the petalinux project directory to open the kernel configuration interface.

```
etalinux-config -c kernel
```

Select NVME Support in Device Drivers.

1.3

Configure root file system tool

To use SSD, you also need to use the file system creation tool. Use the following command to configure the file system and select the file system creation tool in the file system.

```
etalinux-config -c rootfs
```

Go to Filesystem Packages base util-linux and select the option in the following figure.

Enter Filesystem Packages base e2fsprogs and select e2fsprogs-mke2fs. Save and exit the configuration interface.

11.1.4.

Compile the petalinux project and then package BOOT.bin. Copy image.ub, BOOT.bin, and boot.scr from the petalinux project image/linux path to the FAT partition of the SD card. Unzip rootfs.tar.gz to the EXT partition of the SD card. For specific operations, please refer to "Chapter 8 SD Card Root File System" in this tutorial.

2.

Set partition

a.

Insert the SD card you just made into the development board, then insert the SSD and start the board. Run the following command to check if the SSD device is loading normally.

```
; /dev/nvme*
```

2.

Operate/dev/nvme0n1 to set partitions for SSD.

```
disk /dev/nvme0n1
```

Enter n to create a new partition, enter p to create a primary partition. The partition number, starting sector, and ending sector are all kept default in this experiment. Press Enter directly.

3.

Enter wq, save the settings and exit.

4.

Use the following command to create a partitioned file system (format partition).

```
ıkfs -t ext4 /dev/nvme0n1p1
```

5.

Create a new folder and use the mount command to mount the partition.

```
ıkdir -p /run/media/nvme0n1p1

ıount /dev/nvme0n1p1 /run/media/nvme0n1p1
```

3.

Read-write speed measurement

After formatting and mounting the SSD, you can use the time and dd commands to read and write speed tests. (In the new Linux system, you need to use the time command to print the time and calculate the speed yourself.)

a.

CD enters the mounted path.

```
d /run/media/nvme0n1p1
```

2.

Sequential write test

```
me dd if=/dev/zero of=./testfile1 bs=100k count=1310
```

Write speed is (1310 * 100)/1024/0.632 = 202.42 M/s

c.

Sequential read test, pay attention to clearing the cache before testing.

```
cho 3 > /proc/sys/vm/drop_caches

me dd if=./testfile1 of=/dev/null bs=100k
```

Read speed is (1310 * 100)/1024/0.378 = 338.43 M/s

d.

Use the dd command to copy files and test read and write speed.

```
cho 3 > /proc/sys/vm/drop_caches


me dd if=./testfile1 of=./testfile2 bs=100k
```

Read and write speed is (1310 * 100)/1024/0.719 = 177.2 M/s

Part 12: Hello World that can be remotely debugged.

1.

Introduction to GDB

GDB, the GNU Project debugger, allows you to see what happens "internally" during program execution, or what a program is doing when it crashes. Its main functions are as follows:

Start the program and specify any content that may affect its behavior, such as environment variables, running parameters, etc

② Make the program stop running under specified conditions

③ Check what happens when the program stops, such as register status, variable values, etc

④ Manually modify various values during program operation, which can more conveniently verify program functionality

The GNU debugging tools under Linux are gdb and gdbserver. Among them, gdb and gdbserver can complete remote debugging of Linux applications on the target board. Gdbserver is a small application that runs on the target board and can monitor the running of the debugged process. It can communicate with gdb on the upper computer through the network. Developers can input commands through gdb on the upper computer to control the running of processes on the target board and view the contents of memory and registers

2.

Introduction to QtCreator

Qt Creator is a cross-platform Integrated Development Environment (IDE) that allows developers to create applications across desktop, mobile, and embedded platforms.

The Qt library we usually refer to is the library we call when developing desktop applications, which will be linked into our running program. Qt Creator, on the other hand, is a tool that helps us develop applications. Therefore, the two can be considered completely different things.

3.

Experimental objectives

① Get to know the Qt Creator project

② QtCreator configuration GDB connection board

③ Move the runnable target program to the board

Pass parameters to the running program

4.

Qt Creator Project

The project file of Qt Creator is xxx.pro, and "QT =" in the file specifies the QT library component that needs to be used. If it is empty, it means that the QT library is not called.

5.

Obtain the IP Address of the Board

1.

Install our development board with an SD card with a Linux system and power it on

2.

View IP address

The IP address of the board can be viewed through the "ifconfig" command. DHCP is enabled by default in the system. If DHCP service is not supported locally, the query result is shown in the following figure.

At this time, you need to manually assign an IP address. You can use the command "iconfig eth0 192.168.1.45 netmask 255.255.255.0". Here we set the IP address of the board to: 192.168.1.45

6.

QtCreator Configure GDB Connection Board

1.

Open the tool settings item in QtCreator (Tools-- > Options).

2.

Click the "Add..." button and select "Generic Linux Device".

3.

Parameter settings are as follows

The password is root by default and needs to be filled in.

4.

Click "Next" and "Finish", and in the test that appears later, the interface after success is as follows

If not successful, the network connection needs to be checked

5.

Go back to the "Device" settings and click "Apply" to complete this setting

6.

Switch settings and configurations as follows

7.

Run Hello World

1.

Open the helloworld project, drag this folder into the virtual machine in the source code package provided in Chapter 12, then run the pro project inside, and click main.cpp compile under Sources inside. After completion, click .pro above.

2.

Compile application

3.

Click the Run button of QtCreator

We can see that the running print information on the right indicates that the program has run and exited.

If the following error is prompted, it is necessary to check whether the board is powered on and entered the Linux system, and whether the network is normal

② If the following error is prompted, you need to clear the compile result and compile it again.

4.

At this point, we check the user directory "/home/root" of the board and find an extra file "helloworld".

5.

We open the pro file of the project

In the red box is the directory where the program is automatically loaded during runtime, which explains why the file helloworld appears in the "/home/root" directory of the development board system

8.

Operating parameter settings

Sometimes, when we run a program, we need to add some running para

parameters at the end. Below we will introduce how to add running parameters to the running program during remote debugging

1.

Open the following settings

The parameter here is set to 10.

2.

Set a breakpoint at the main function if position

3.

Click the debug button or shortcut F5

4.

We see that the program has stopped at the breakpoint we set

5.

Press F5 to continue running, we see the print information as follows

We can see that the starting number after "hello world" has changed to 10, indicating that the parameters we set have taken effect

Part 13: edge detection of OpenCV

Note: From this chapter to Chapter 18, the factory system is required. The specific steps are as follows:

Find this folder and extract it, then extract this file:

Then refer to the following PDF document to create a factory system.

There will be configuration for the first power-on. Execute the following statement and then reset the development board to enter the system faster.

```
ɲ -rf ./*
```

1.

Introduction to OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source Computer Vision and Machine Learning software library. OpenCV was established to provide a universal infrastructure for Computer Vision applications and accelerate the application of

machine perception in commercial products. As a BSD-licensed product, OpenCV makes it easy for enterprises to utilize and modify code.

The library has over 2500 optimization algorithms, including a complete set of classic and state-of-the-art Computer Vision and Machine Learning algorithms. These algorithms can be used to detect and recognize faces, recognize objects, classify human behavior in videos, track camera movements, track moving objects, extract 3D models of objects, generate 3D point clouds from stereo cameras, stitch images together to generate high-resolution images of the entire scene, find similar images from image libraries, remove red-eye from images taken with flash, track eye movements, identify scenery and establish markers to cover augmented reality, and more. OpenCV has a community of over 47,000 users and is estimated to have over 18 million downloads. It is widely used by companies, research groups, and government entities.

Many implementations of Xilinx's reVision are based on the OpenCV interface, with the hardware implementation being xfopencv. This will help us achieve hardware (FPGA) acceleration of the corresponding algorithm.

2.

Introduction to Edge Detection

Edge detection is a fundamental problem in image processing and Computer Vision. The purpose of edge detection is to identify points in digital images where brightness changes significantly. Sig

1.

Introduction to OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source Computer Vision and Machine Learning software library. OpenCV was established to provide a universal infrastructure for Computer Vision applications and accelerate the application of machine perception in commercial products. As a BSD-licensed product, OpenCV makes it easy for enterprises to utilize and modify code.

The library has over 2500 optimization algorithms, including a complete set of classic and state-of-the-art Computer Vision and Machine Learning algorithms. These algorithms can be used to detect and recognize faces, recognize objects, classify human behavior in videos, track camera movements, track moving objects, extract 3D models of objects, generate 3D point clouds from stereo cameras, stitch images together to generate high-resolution images of the entire scene, find similar images from image libraries, remove red-eye from images taken with flash, track eye movements, identify scenery and establish markers to cover augmented reality, and

more. OpenCV has a community of over 47,000 users and is estimated to have over 18 million downloads. It is widely used by companies, research groups, and government entities.

Many implementations of Xilinx's reVision are based on the OpenCV interface, with the hardware implementation being xfopencv. This will help us achieve hardware (FPGA) acceleration of the corresponding algorithm.

2.

Introduction to Edge Detection

Edge detection is a fundamental problem in image processing and Computer Vision. The purpose of edge detection is to identify points in digital images where brightness changes significantly. Significant changes in image attributes usually reflect important events and changes in attributes. These include discontinuities in depth, discontinuities in surface orientation, changes in material attributes, and changes in scene lighting. Edge detection is a research field in image processing and Computer Vision, especially in feature extraction.

Image edge information is mainly concentrated in the high-frequency band. Usually, image sharpening or edge detection is essentially high-frequency filtering. We know that differential operation is to calculate the rate of change of the signal, which has the

function of strengthening the high-frequency component. In spatial operations, image sharpening is to calculate differentiation. Due to the discrete signal of digital images, differential operation becomes calculating difference or layer. There are various edge detection (layer) operators in image processing, commonly used including ordinary first-order difference, Robert operator (cross-difference), Sobel operator, etc., which are based on finding layer strength. Laplace operator (second-order difference) is based on zero-crossing detection. By calculating layers and setting thresholds, edge images are obtained.

The general steps for edge detection are:

① Filtering: Edge detection algorithms are mainly based on the first and second derivatives of image intensity, but derivatives are sensitive to noise, so filters are needed to improve the performance of edge detectors related to noise

② Enhancement: The basis of enhancing the edge is to determine the change value of the neighborhood intensity of each point in the image. The enhancement algorithm can highlight the points with significant changes in the neighborhood intensity value of the canary release point

③ Detection: There are many points in the neighborhood with large layer values, but in specific applications, these points are not the edge points to be found and need to be selected

3.

Experimental objectives

① Use OpenCV to capture USB camera images

Edge detection of canny, sobel, and laplacian algorithms

③ Use OpenCV to display images

④ Response to OpenCV window closing operation

4.

Software flowchart

5.

Operational readiness

1.

DP interface connected to the display

2.

Plug in the usb camera

3.

After the development board is powered on and the system start-up is completed, the dp monitor can correctly display the boot desktop

4.

At the end point, you can view the USB camera device (after viewing, remember your camera device number).

6.

Program running

1.

You can refer to the previous chapter routine, open the 13_opencv_edgedete project, use QtCreator to run the application directly, the running effect is as follows (if you run an error, please see the second point)

2.

If the following error is reported, modify it:

Change the camera device number to your own device number

7.

Code parsing

1.

Set environment variables

Enter the following command in the end point, which is equivalent to calling setenv in the code.

```
xport DISPLAY=:0.0
```

2.

Launch the desktop

```
etc/init.d/xserver-nodm start
```

3.

Display resolution adjustment

As in the above experiment, using a 4K monitor, the icons on the startup desktop are very small. In order to achieve better display effect, in the application, set the display output resolution to 1080P through system calls. The command is as follows:

```
randr --output DP-1 --mode 1920x1080
```

4.

Display output management

The system defaults to using output power management, which means that after a period of time, the monitor will go black. At this time, we can use a command to cancel this function. The command is as follows:

```
set s 0 0

set dpms 0 0 0
```

5.

Multithreaded operation

This experiment uses three different algorithms to complete edge detection, and the three algorithms are assigned to three thread tasks, which can fully utilize ARM's quad-core A53.

6.

USB camera capture

The camera resolution is set to 640x480, and the default camera device is/dev/video0. This is because the algorithm is a bit slow and the displayed image may have a slight delay.

7.

Canny Edge Detection

John F. Canny developed a multi-level edge detection algorithm in 1986, which is considered by many to be the *optimal algorithm* for edge detection. The three main evaluation criteria for optimal edge detection are:

- Low error rate: Identify as many actual edges as possible while minimizing false positives generated by noise.
- High positioning: The identified edges should be as close as possible to the actual edges in the image.
-

Minimum response: An edge in an image can only be identified once.

Cv:: canny function parameters are as follows:

nputArray	nage,	put image
)utputArray	dges	utput edge image
ouble	reshold1	hreshold 1

ouble	hreshold2	hreshold 2
ıt	ɔertureSize=3	obel operator size
ool	2gradient=false	/hether to calculate the image yer in a more accurate way

8.

Sobel Edge Detection

This is a commonly used edge detection method. The function of the Soble operator combines Gaussian smoothing and differential differentiation, also known as the first-order differential operator. The derivative operator takes the derivative in both horizontal and vertical directions to obtain a layer image in the X and Y directions. Disadvantages: It is sensitive and easily affected, and noise reduction needs to be achieved through Gaussian blur (smoothing). The accuracy of edge positioning is not high enough.

Cv:: The parameters of the sobel function are as follows:

ıputArray	˙c,	ıput image

)utputArray	st	utput edge image
ıt	depth	utput image depth
ıt	X	he order of the derivative x.
ıt	Y	rder of derivative y

ıt	size = 3	he size of the Sobel kernel
ouble	cale = 1	ptional scaling factor for alculated derivative values
ouble	elta = 0	ptional incremental values dded to the results before oring them in dst.

ıt	orderType=BORDER_DEFAUL	ixel extrapolation

9.

Laplacian Edge Detection

The Laplacce operator is a second-order differential operator in n-dimensional Euclidean space. It is more suitable when only caring about the position of the edge without considering the canary release difference of the pixels around it. The response of the Laplace operator to isolated pixels is stronger than that of edges or lines, so it is only suitable for noise-free images. In the presence of noise, low-pass filtering is required before using the Laplacian operator to detect the edge. Therefore, the usual segmentation algorithm combines the Laplacian operator and the smoothing operator to

generate a new template. The Laplace operator, like the Sobel operator, belongs to the spatial sharpening filtering operation.

Cv:: Laplacce function parameters are as follows:

nputArray	rc,	put image
utputArray	st	utput edge image
nt	depth	utput image depth

ıt	size = 1	sed to calculate the size of the econd derivative kernel
ouble	cale = 1	ptional scaling factor for alculating Laplacce
ouble	elta = 0	ptional incremental values dded to the results before oring them in dst.

ıt	orderType=BORDER_DEFAUL	ixel extrapolation

Part 14: OpenCV + Qt face detection

1.

Cascade Classifier for OpenCV

A classifier is a device that determines whether something belongs to a certain category. Cascade classifier can be understood as concatenating N single-class classifiers. If an object can belong to all the classifiers in this series, the final result is determined to be yes. If there is any discrepancy, it is determined to be no. For example, a human face has many attributes. We make each attribute into a classifier. If a model meets all the attributes of a human face that we define, then we consider it a human face. So what do these attributes refer to? For example, a human face needs to have two eyebrows, two eyes, a nose, a mouth, a roughly U-shaped chin or contour, and so on.

Part 18: linux register operation

1.

Linux memory mapping

Memory mapping refers to mapping a memory area from user space to kernel space. After successful mapping, modifications made by users to this memory area can be directly reflected in kernel space. Similarly, modifications made by kernel space to this

area also directly reflect user space. This is very efficient for transferring large amounts of data between kernel space and user space.

For example, when reading a file from a hard drive into memory, the data needs to be copied through the file system, and the data copying operation is implemented by the file system and hardware driver. In theory, the efficiency of copying data is the same. However, accessing files on the hard drive through memory mapping is more efficient than using read and write system calls. Why is this? The reason is that read () is a system call that copies data. It first copies the file content from the hard disk to a buffer in kernel space, and then copies the data to user space. In this process, it actually completes two data copies. Mmap () is also a system call, and there is no data copy in mmap (). The real data copy is performed during page fault interrupt processing. Since mmap () maps the file directly to user space, the interrupt processing function copies the file directly from the hard disk to user space based on this mapping relationship, only one data copy is performed. Therefore, the efficiency of memory mapping is higher than that of read/write.

2.

Introduction to the experiment

Sometimes, the function of a certain peripheral has been verified under bare metal. If the peripheral does not use interrupts, we can use the method of mapping registers under Linux to directly port the program under bare metal to Linux for use. This way, we can save the trouble of developing drivers.

This routine transplanted vitis under gpio routine, drive ps_led, for flashing once per second action. Other operations, the process is similar.

3.

Run the program

After running the application register_opt, you can see that the PS LED flashes once per second

4.

Code analysis

1.

Regarding the register address of GPIO, we copied it from the GPIO routine of Vitisi. Our development strategy should also be to verify it with Vitisi's bare metal program

first. For many peripherals and FPGA-side IPs, Vitisi will help us generate the method and address of the operation, so we don't need to look for corresponding relationships.

2.

When opening/dev/mem, use the options O_SYNC

After writing data to the outside, the data is usually written to the cache buffer. O_SYNC will ensure that the data is written to the peripheral device before

Part 19: boot the kernel from QFlash and mount the root file system from EMMC

Our development board does not support booting directly from EMMC, but we can use the solution of booting from QFlash, loading the kernel from EMMC, and mounting the root file system. Loading the kernel from EMMC and mounting the root file system and SD card are similar, but unlike SD cards, EMMC cannot be formatted directly on a PC and copied into the system. Therefore, we need to use other methods to boot a system, create an EMMC partition in this system, and copy and decompress the kernel and root file system into EMMC.

1.

Create and configure a petalinux project

1.

Create a new folder "linuxQflashAndEmmc" in the path "/home/alinx/peta_prj".

```
etalinux-create -t project -n petalinux --template zynqMP
```

Open the end point in this folder, use the above command to create a new petalinux project, and create a hardware folder to store the hardware description file (use the vivado project named ps_hello in "Vitis Application Tutorial" to compile the xsa file).

2.

Enter the petaLinux project folder in the end point, use the following command to configure the hardware description file of the petalLinux project, and open the configuration interface. Do not configure anything and exit directly.

```
etalinux-config --get-hw-description ../hardware/
```

4.

Select primary sd in Subsystem AUTO Hardware Settings --- > Advanced bootable images storage Settings --- > kernel image settings --- > image storage media option.

5.

Subsystem AUTO Hardware Settings --- > Flash Settings, follow the figure to modify the partition size of QSPI flash.

6.

Select'EXT4 'for'Image Packaging Configuration --- > Root filesystem type'.

.

In the tutorial project, emmc is mmc0 because only one partition is needed to store the file system. Therefore, we fill in "/dev/mmcblk0p1" in the "Device node of SD device" option.

8.

Add an "ext4" in the "Root filesystem formats" option.

9.

Configure offline compile as needed. The "Yocto Settings" interface configuration is as follows. Then save the configuration and exit the configuration interface.

10.

Use the petalinux-config -c u-boot command to configure u-boot. After opening the configuration interface, enter the ARM architecture --- > Boot script offset interface and set the offset address of the bootscr partition to 0xA10000.

Save and exit.

1.

Open the "boot.cmd.default.initrd" file in the path of "/project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr" in the petalinux project, and add the following two lines of content, as shown in the figure below.

```
etenv bootargs "earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/mmcblk0p2 rw
ɔotwait"

ɜtload mmc 0:1 0x10000000 image.ub && bootm 0x10000000
```

12.

Modify the device tree.

```
nclude/ "system-conf.dtsi"

{

 SD */
```

12.

Modify the device tree.

```
nclude/ "system-conf.dtsi"

{

' SD */

:sdhci1 {

isable-wp;

o-1-8-v;
```

```
:sdhci0 {

isable-wp;

o-1-8-v;









' USB */

:dwc3_0 {

tatus = "okay";

r_mode = "host";
```

13.

After compiling the petalinux project, copy image.ub and rootfs.tar.gz from the image/linux path of the petalinux project to the NFS shared path.

14.

To package BOOT.bin, you need to include u-boot, fsbl, and boot.scr. Use the following command

```
etalinux-package --boot --u-boot --add ./images/linux/boot.scr --offset 0xA10000 --load
x20000000 --force
```

15.

Burn BOOT.bin to the flash of the development board. Refer to the Vitis application tutorial for the burning method (don't worry about it after burning, continue to do the following).

2.

Set up EMMC partition to make root file system

Use a non-Qspi flash way to boot the development board. In this experiment, use the system created in "Chapter 11 Linux NVMe SSD Operation" to boot the system development board (copy BOOT.bin, image.ub, and boot.scr from Chapter 11 project to the FAT partition of the SD card; extract rootfs.tar.gz to the EXT partition of the SD card) and set the EMMC partition.

1.

Use the command fdisk /dev/mmcblk0 to set the partition, and if there is already a partition, use the command d to delete the existing partition.

2.

First, enter n to create a new partition to store the kernel. Keep the partition number and starting sector in the default settings and press Enter directly. Enter + 1G to set the ending sector.

3.

Enter n again to create a second partition for the file system, and keep all settings default.

4.

Then enter wq to save and exit the partition settings.

5.

Format the first partition of EMMC to fat32 and the second partition to ext4 using the following command.

```
ıkfs.vfat /dev/mmcblk0p1

ıkfs.ext4 /dev/mmcblk0p2
```

6.

If the system does not automatically mount after the partition is set up, manually mount the two partitions of EMMC to/media/sd-mmcblk0p1 and/media/sd-mmcblk0p2 respectively. If these two folders do not exist, create them manually.

```
ıkdir /media/sd-mmcblk0p1

ıkdir /media/sd-mmcblk0p2

ıount /dev/mmcblk0p1 /media/sd-mmcblk0p1/
```

```
ıount /dev/mmcblk0p2 /media/sd-mmcblk0p2/
```

7.

Mount the nfs shared path in the virtual (the command to mount the shared path needs to be modified according to the actual). Copy the image.ub in the shared path to/media/sd-mmcblk0p1.

```
ıount -t nfs -o nolock 192.168.0.252:/home/alinx/work /mnt

p /mnt/image.ub /media/sd-mmcblk0p1/
```

8.

Use the following command to extract rootfs.tar.gz from the shared path to/media/sd-mmcblk0p2.

```
ar -zxvf /mnt/rootfs.tar.gz -C /media/sd-mmcblk0p2/

ync
```

8.

Use the following command to extract rootfs.tar.gz from the shared path to/media/sd-mmcblk0p2.

```
```

```
ar -zxvf /mnt/rootfs.tar.gz -C /media/sd-mmcblk0p2/

ync
```

3.

Qflash + EMMC startup test

Set the development board to Qspi flash to start and power on. The system can be seen to start successfully through the PS_UART serial port.